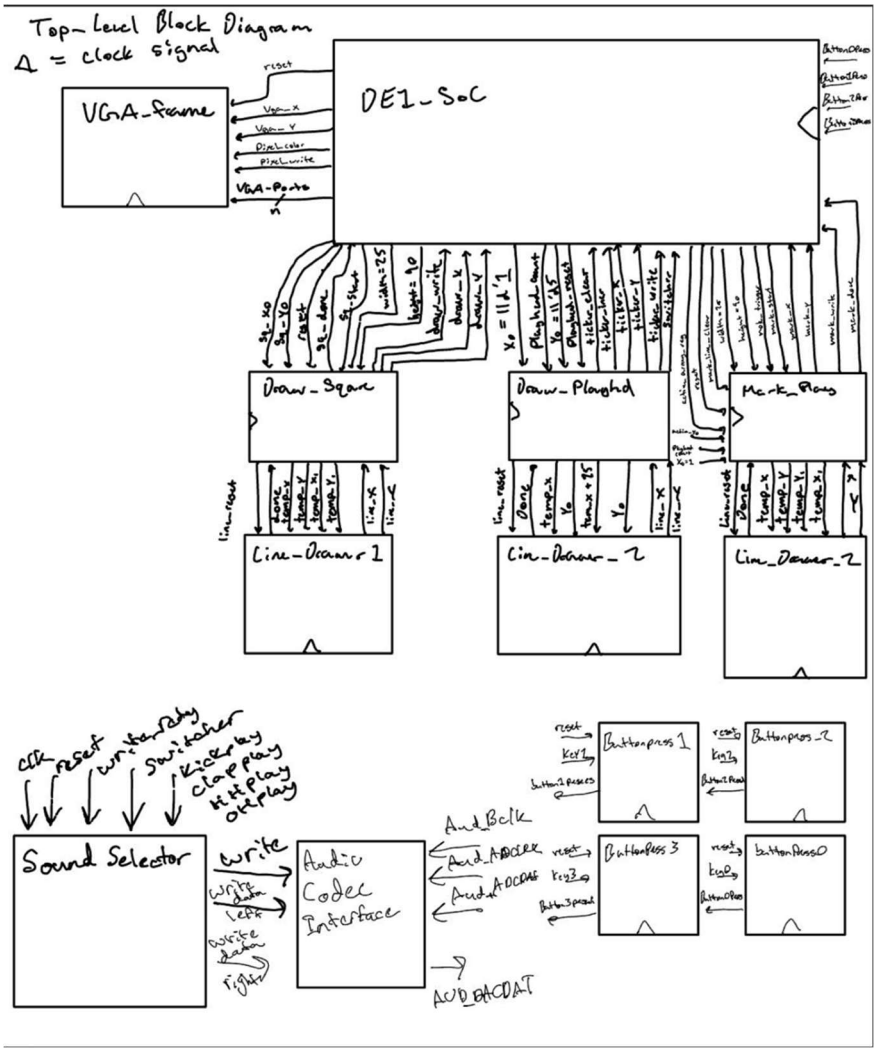
Design Procedure

For our final project for EE 371, we decided to create a beat sequencer using the audio codec and VGA display driver to create an interactive display. The user has four inputs: KEY0, KEY1, KEY2, and KEY3, each corresponding to a row of blank boxes on the display. The first row plays the kick drum, second plays the clap, third plays the open hat, and fourth plays the high hat. The system runs at 130 bpm and there is a ticker over the boxes that moves left to right at 130bpm. If the user wants to place a beat to be played, they must press the corresponding key to the note where the ticker is currently at. If pressed, it will mark the rectangle letting the user know that a note will be played there when the ticker is over the box. Two keys cannot be pressed at once. Similarly to adding a note, the user can remove the note by pressing the key again over a marked box which will remove the sound and mark.

To implement this system, we had to create multiple modules and connect them all through the top level DE1_SoC. A square_drawer module was created to draw the boxes on the screen while draw_playhead module was created to simulate the bpm and draw the ticker going over the boxes at the proper bpm. When a user pressed a key, to mark the box a mark_play module was created to draw and remove the line across the box at the time of ticker playhead. All the drawings were implemented using the Line_Drawer and VGA display modules from lab 5. To play the sound we read data from roms which have our noises and if played the noise is outputted to an audio codec. We play multiple noise by extending the bit to 26 and then shrinking to 24 in the sound_selctor module.

Overall System: Top Level Block Diagram



Results

Square_Drawer

The square_drawer ASMD in figure 2 describes the implementation of the module in Verilog. There is an internal counter that iterates from 0-4 telling the machine which line to draw for the rectangle. Count==0 acts as the start trigger and it runs through the lines at each iteration of count. The module instantiates the line_drawer algorithm with temporary values given from the square_drawer shown in figure 3. The module also has signals to feed to the VGA such as pixel_write, x, and y, which are used in the DE1_SoC to draw the actual lines that are being created.

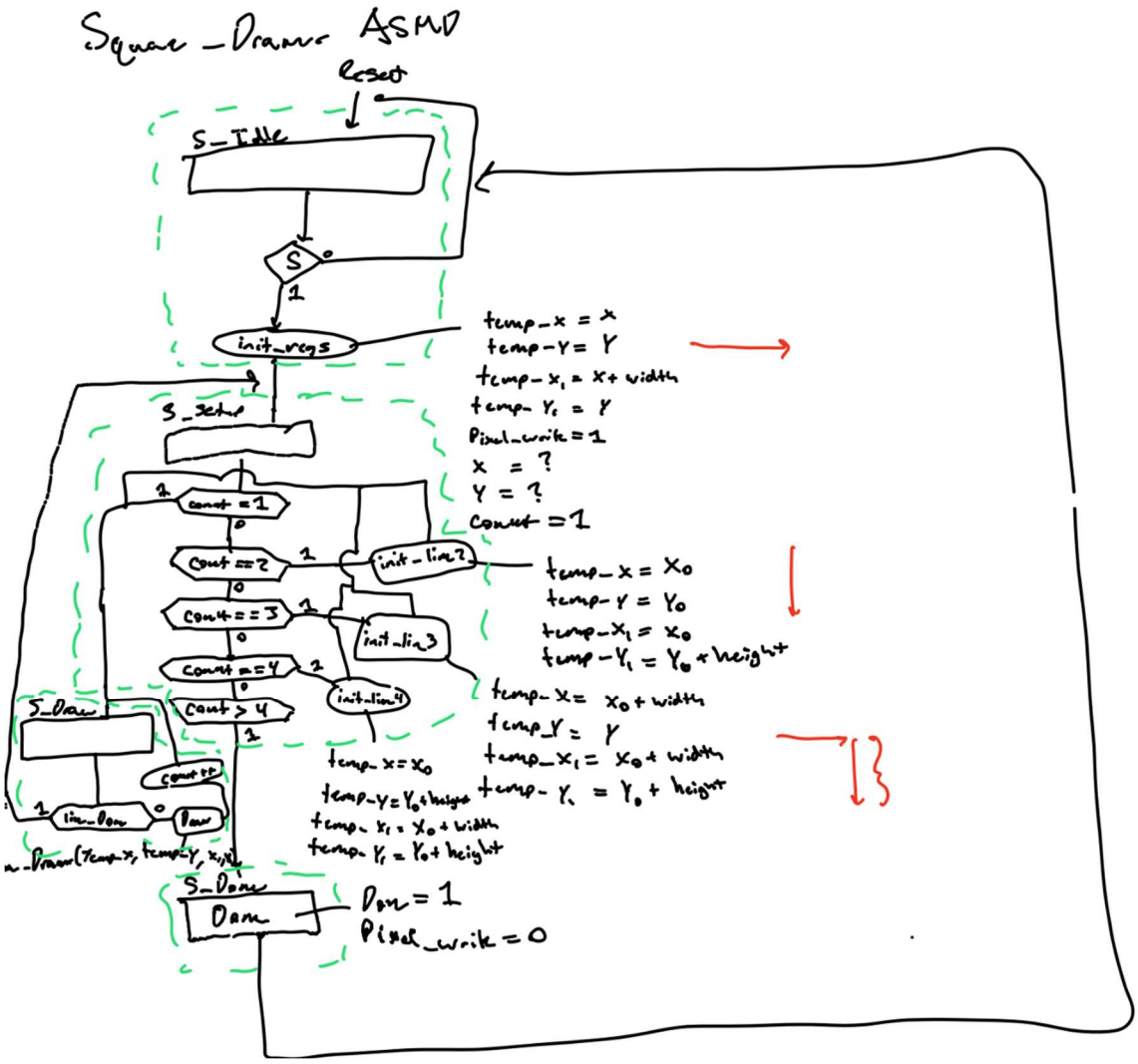


Figure 2: ASMD for Square_Drawer module showing all datapath logic

```
always_comb begin
    pixel_write = (ps == S_Draw);
    done        = (ps == S_Done);

    // To draw a new line during the setup phase
    line_reset  = (ps == S_Setup);
end

//logic to change the coordinates
always_ff @(posedge clock) begin
    if (reset) begin
        ps      <= S_Idle; //init regs
        count   <= 3'd0;
        temp_x  <= 11'd0;
        temp_y  <= 11'd0;
        temp_x1 <= 11'd0;
        temp_y1 <= 11'd0;
    end else begin
        ps <= ns;

        if (ps == S_Idle && start == 1) begin
            temp_x  <= x0;
            temp_y  <= y0;
            temp_x1 <= x0 + width;
            temp_y1 <= y0;
            count   <= 3'd1; // Initialize count to 1 for first line
        end
        else if (ps == S_Setup && count == 3'd1) begin //first edge
            temp_x  <= x0;
            temp_y  <= y0;
            temp_x1 <= x0 + width;
            temp_y1 <= y0;
        end
        else if (ps == S_Setup && count == 3'd2) begin //second line
            temp_x  <= x0 + width;
            temp_y  <= y0;
            temp_x1 <= x0 + width;
            temp_y1 <= y0 + height;
        end
        else if (ps == S_Setup && count == 3'd3) begin //third line
            temp_x  <= x0 + width;
            temp_y  <= y0 + height;
            temp_x1 <= x0;
            temp_y1 <= y0 + height;
        end
        else if (ps == S_Setup && count == 3'd4) begin //fourth line
            temp_x  <= x0;
            temp_y  <= y0 + height;
            temp_x1 <= x0;
            temp_y1 <= y0;
        end

        //count will only increment when the line is done drawing
        if (ps == S_Draw && line_done) begin
            count <= count + 3'd1;
        end
    end
end
```

Figure 3: DataPath for Square_Drawer showing the coordinate and output wires

Draw_Playhead

The ASMD for the Draw_Playhead module is shown in figure 4 below. There is a counter different from the counter in square_module that counts from 0-15, corresponding to the current box that the playhead is above. If count == 0 acts as a start signal. It will draw the playhead over the current box where the count is. When the line is done drawing it will wait until the internal bpm counter reaches its final value and then will erase the line, increment count, and draw the next line. The code in figure 5 shows how the bpm counter increments inside a flipflop and how the playhead increments its position by adding the top coordinate to the width times the count number.

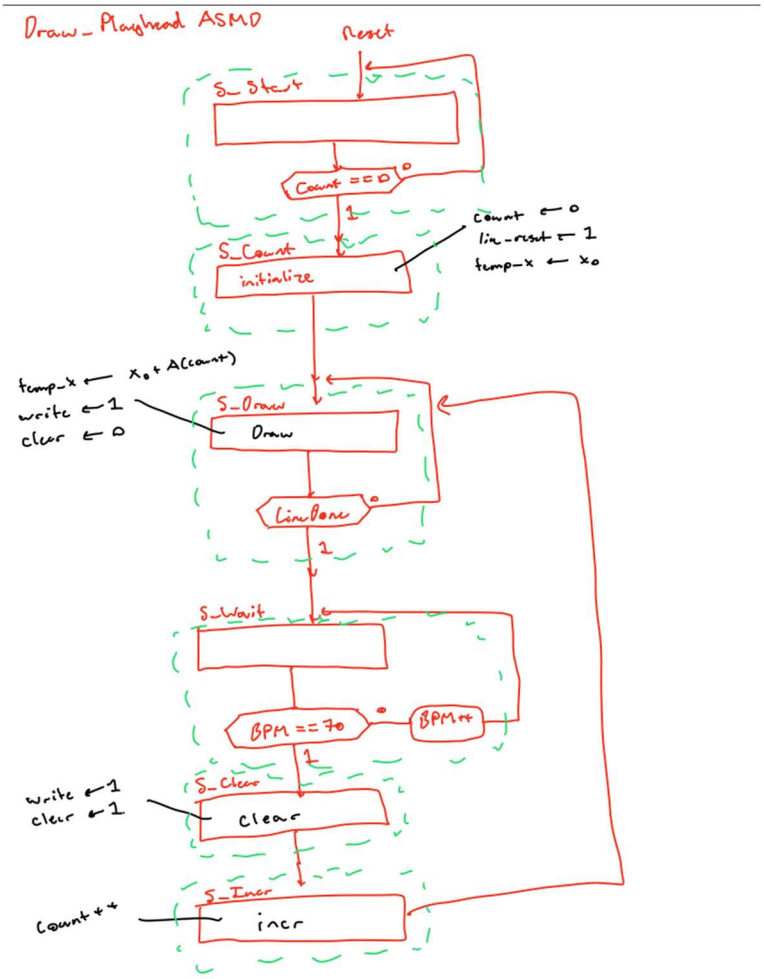


Figure 4: Draw_Playhead ASMD

```
    always_ff @(posedge clock) begin
       if(reset) begin
          ps <= S_Start;
          bpm_counter <= '0;
       end
       else begin
          ps <= ns;

          //memory needed to increment the bpm counter
          if(ps == S_Wait) begin
             bpm_counter <= bpm_counter + 26'd1;
          end
          else bpm_counter <= '0;
       end
    end


else if(ps == S_Draw) begin
   temp_x= x0 + (11'd40 * {7'b0, count});
   pixel_write = 1'b1;  //tell the vga to start drawing
   line_clear = 1'b0;  //set color to white
   switcher = 1'b0;
end
```

Figure 5: Playhead code that shows inside the flip flop and how coordinates are handled

Mark_Play

The ASMD in Figure 6 for the Mark_Play module describes its implementation. Once the start is triggered, it will initialize all of the wires. Then it will stay in the wait state until a button is pressed. Once a button is pressed it will either erase the line or draw the line on the current box. This is based on weather array_reg[count] == 0 or not. If it is zero that means there is no line on the current box, and if it is 1, then that means that there is a line on the current box. When the line is done drawing, it will loop back to the wait state and wait for the next key to be pressed. In Figure 7, the datapath for drawing the line is shown. When line_clear is zero, that will send to the VGA to draw a white line, and black otherwise. Similarly to the the Draw_Playhead, the temp_x0 coordinate is calculated based on the count and the width of the boxes.

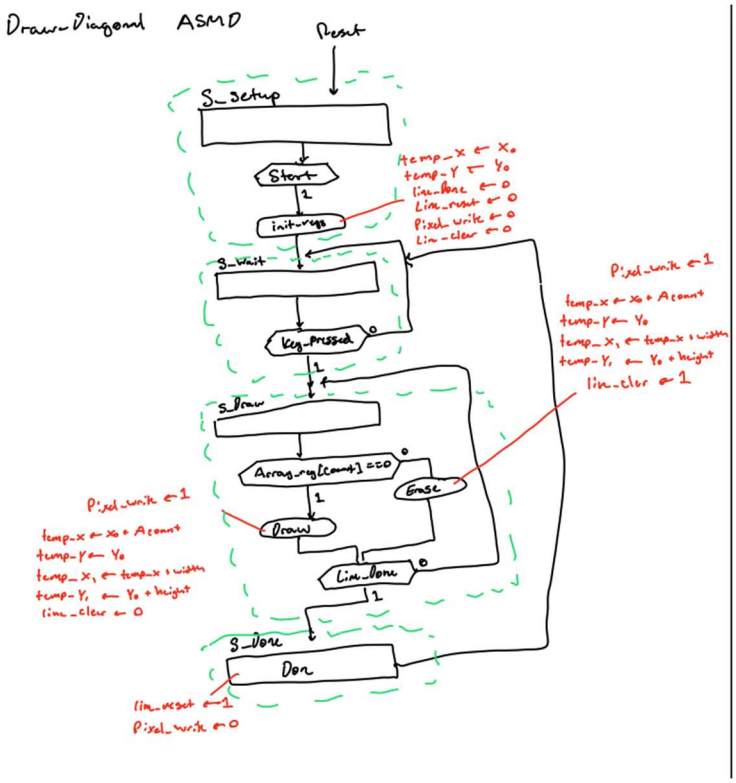


Figure 6: ASMD for Mark_Play module

```
else if(ps == S_Draw) begin
   if(array_reg[count] == 0) begin
      pixel_write = 1'b1; //draw
      temp_x0 = x0 + (11'd40 * {7'b0,count});
      temp_y0 = y0;
      temp_x1 = temp_x0 + width;
      temp_y1 = y0 + height;
      line_clear = 0; //white
   end
   else begin
      pixel_write = 1; //draw
      temp_x0 = x0 + (11'd40 * {7'b0,count}); //current box top left x coordinate
      temp_y0 = y0;
      temp_x1 = temp_x0 + width;
      temp_y1 = y0 + height;
      line_clear = 1; //black , erase
   end
end
else if(ps == S_Done) begin
   line_reset = 1'b1;
   pixel_write = 0;
   all_done = 1'b1;
end
```

Figure 7: Datapath logic showing when to draw/erase a line

soundSelector

This is the module which reads the roms of all the sounds and combines into a writedata (left and right) to go into Audio Codec. It works by reading all the rom for each input and if the play is indicated for the noise it adds it to selector. To do this it we extended it into 26 bits and added the samples on top of each other. Then it gets turned back to 24 bits were if it too quiet gets lowest value and too big gets highest value.Netx the addr for each of the roms are updated. Once at the end it stays there until selector turns on (an input meaning next clock cycle is new beat) and then addr for the sounds go to 0. We then send a write signal if any sound has been played and the 24bit noise as the left and right write data.

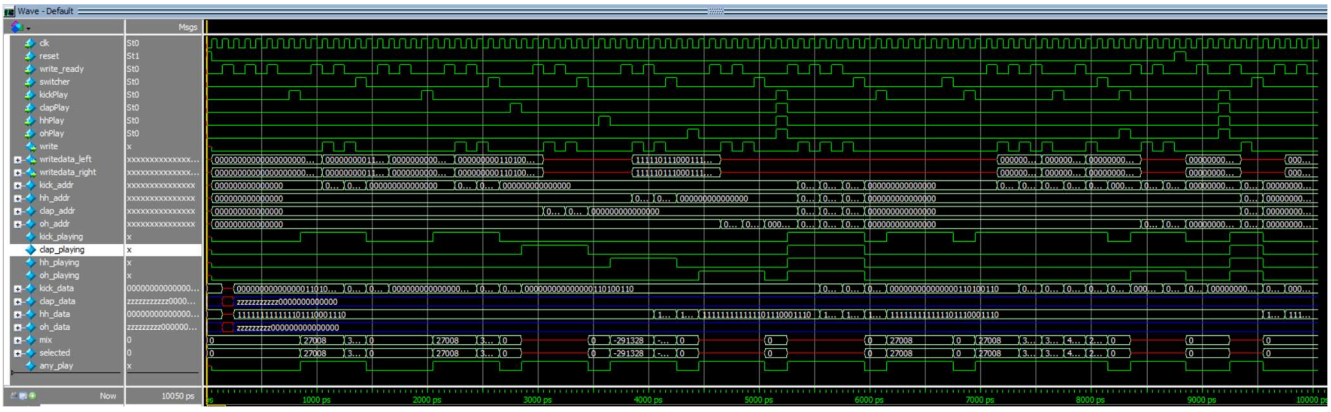


Figure 8: ModelSim for soundSelector

Above is the Simulation for soundSelector. The first case we tested is rest. Then we tested no sound. We can see that the addr stay at 0 along with mix and selected. Any play also stays at 0. Next we triggered the kick. This caused write to go to one, and the kickplay to begin advancing. Next we turned on switcher. This caused this made write go to 0 and the kick to stop. Next we retriggered the kick after switcher which caused write ti become one and addr to restart from 0. Next we tested the rets if the sounds (you can see the individual plays going to one) and reset them by turning selector on then off. Next we tested all at once around 5100ps. Then we

checked what would happen if write_ready was never asserted. We can see that addr don't advance as expected. Next we retriggered our playback. This made our addr rest back to 0 as we need to loop back. Finally, we tested all four of our instruments with write -0 causing all four to stop as needed.

Square_Drawer

Below in figure 11 is the modelsim for the square drawer module. It iterates through every state and shows that it draws each edge, triggering the line_done signal after every width and height. The simulation below only shows the first two lines being drawn. The first x and y coordinates are given and the first line begins drawing. Once it is done, line_done asserts high and the x and y coordinates change to draw the second line.

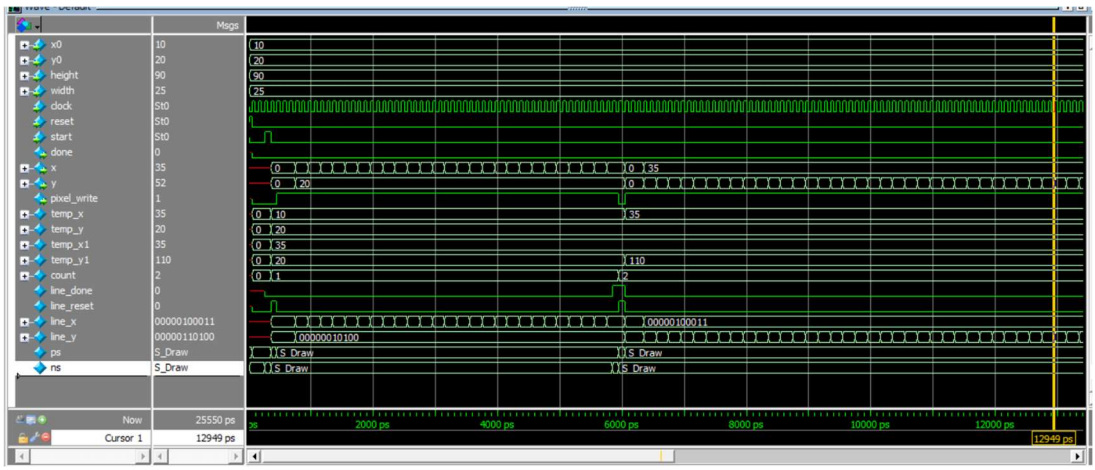


Figure 9: Modelsim for Square_Drawer Module

Draw_Playhead

The modelsim for Draw_Playhead is shown in figure 12 below. Count is zero so it transitions to S_Count, then it transitions to S_Draw. It will stay in the draw state and transition to the wait state until the bpm checker is asserted. For the modelsim, the bpm counter only counts up to 4. It then advances to S_Clear to wipe the current playhead. Once that is done it transitions to S_Incr and incr_count is asserted.

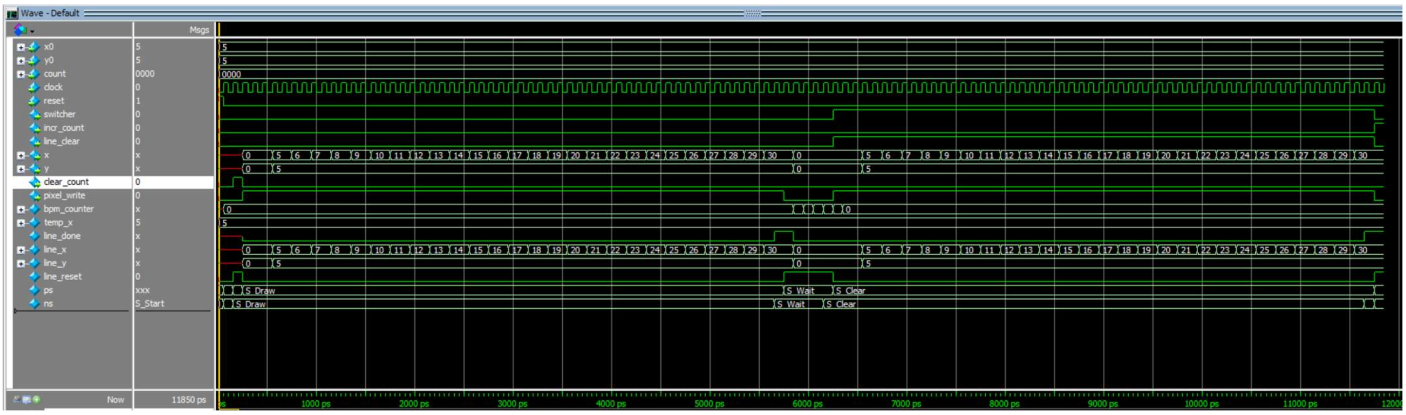


Figure 10: Modelsim for Draw_Playhead module

Mark_Play

The modlsim for Mark_Play is shown in figure 13 below. It shows the coordinates for a line that is drawn and array_reg being 1. Then all_done is asserted, button press is triggered and the x and y coordinates change to the value that corresponds to the value of count. Since it array_reg is 1, that means that the line was drawn in the simulation before. Now, line_clear is asserted high and it starts drawing a black line over the current white line.

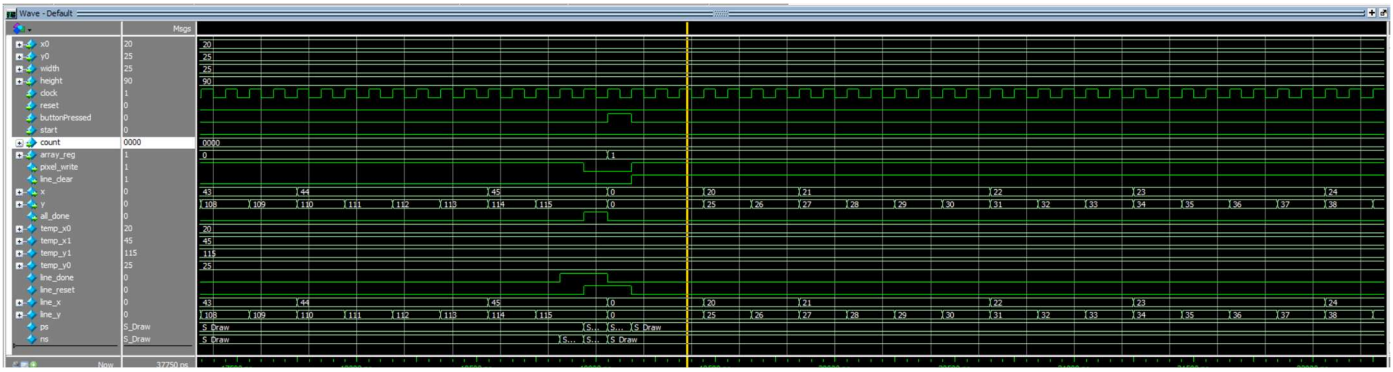


Figure 11: Mark_Play modelsim

buttonPress

This module is to handle the metastability that occurs with having two buttons. All it does is when the button pressed it aligns it as a next state condition making sure it stays on clock cycle. To test this we check for a button being pressed one clock cyle and helkd down(Figure 14).

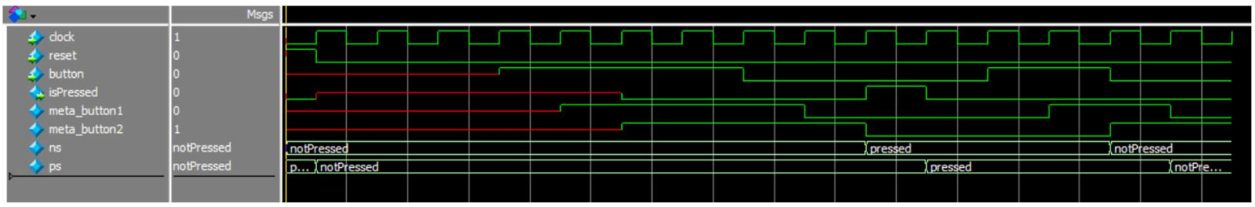


Figure 12: buttonPress ModelSim

Line_Drawer

The modelsim in Figure X-Y show the simulation for all cases of our Line Drawer module. It works by using our alogrthim and going from the smaller x cord between the start and ending point. X0 and y0 are our start x1 and y1 is the end. It is important to note that in cases 4 and 6 it appears x is derc but that is because it is steep so there is a swap in the x cords and y cords. Basically, x_curr always increments to N regardless of case it is just for some steep cases the direction swap is not visible but is for others. We also can see in all cases that error is updating so that our equation is satisfied. Error works by checking how far the ideal line has drifted from the pixel it is on. We use deltax as the centers rounding so there is no bias up or down. Next we update y curr and the error and check next if it should be changed. Finally, once the x and y are in the specified location (start to end) done is asserted with bost ps and ns going to S_DONE.

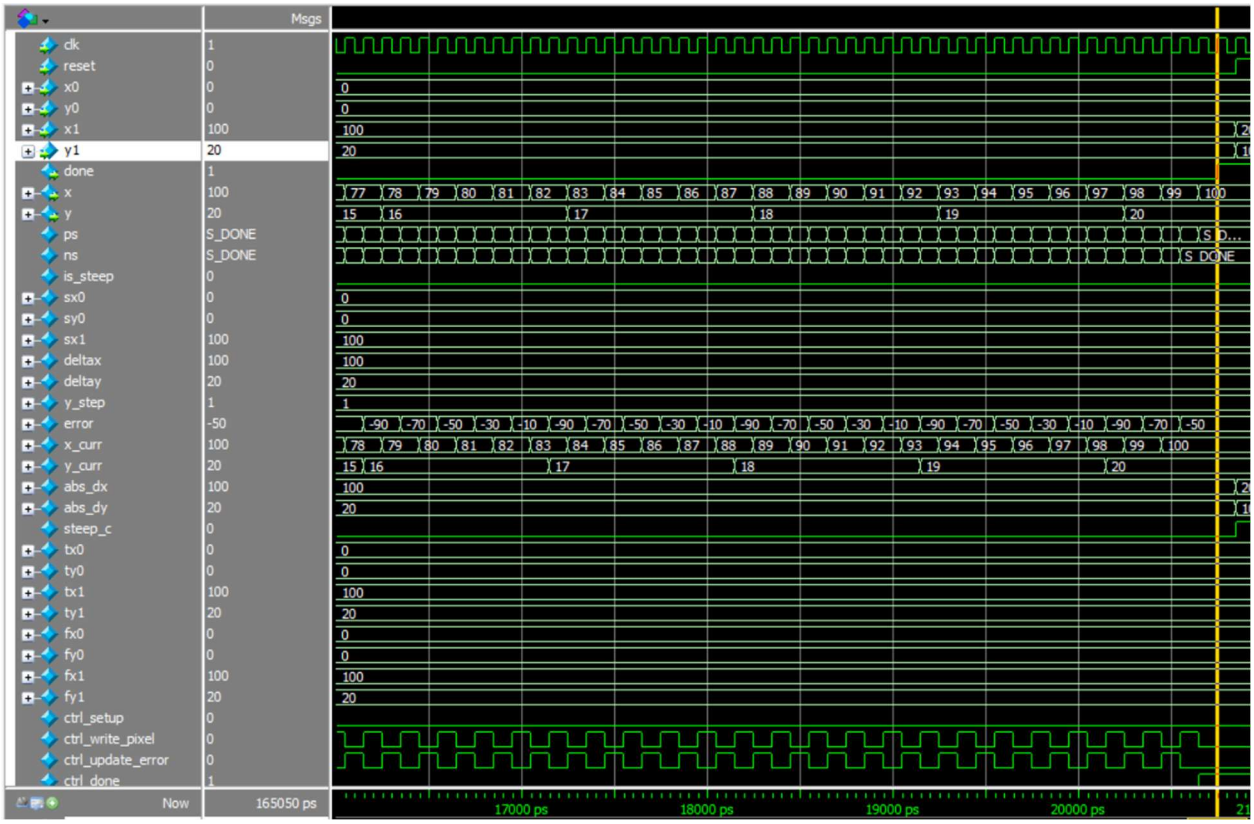


Figure 13: Case 1: Down Right Gradual

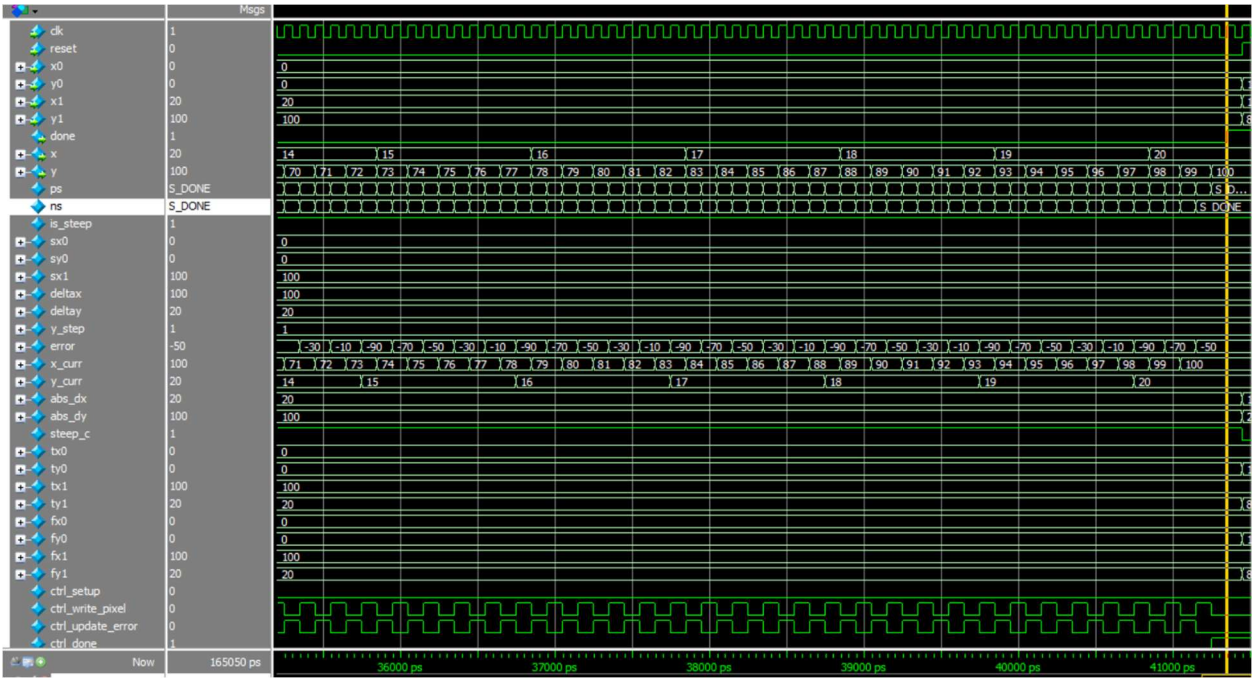


Figure 14: Case 2: Down Right Steep

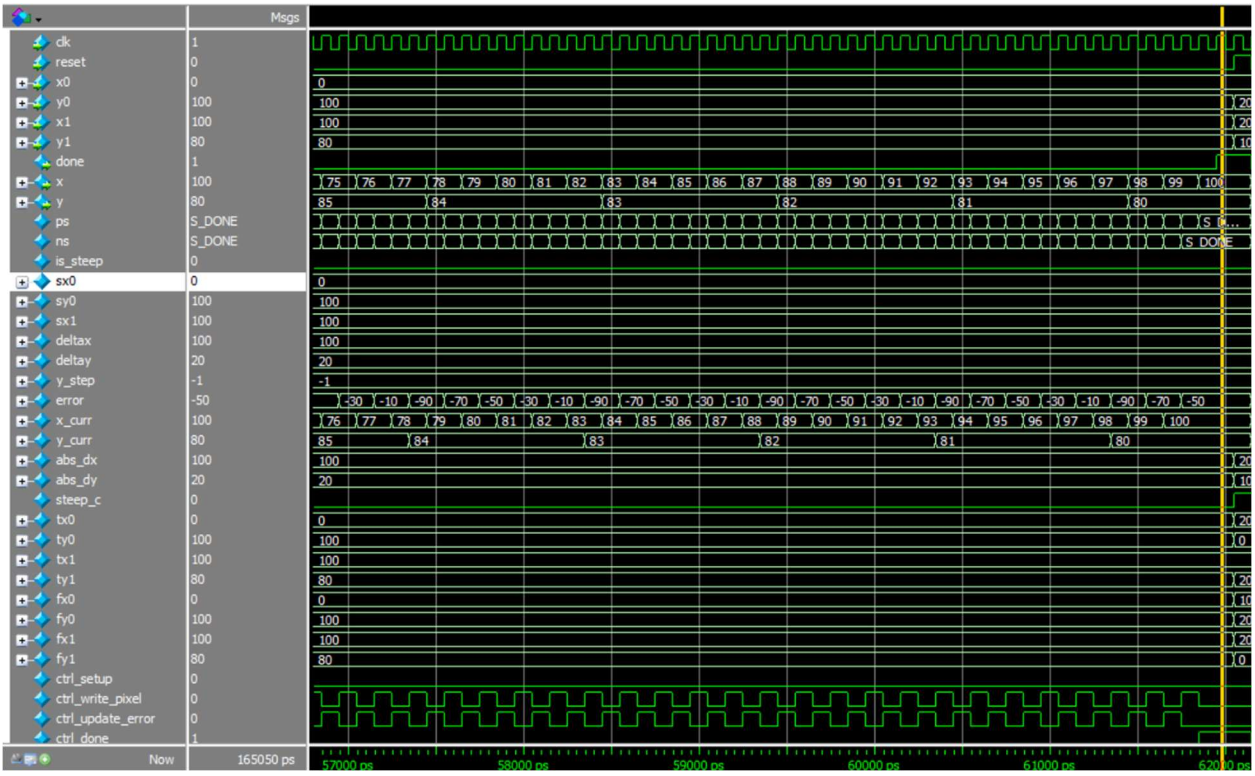


Figure 15: Case 3: Up Right Gradual

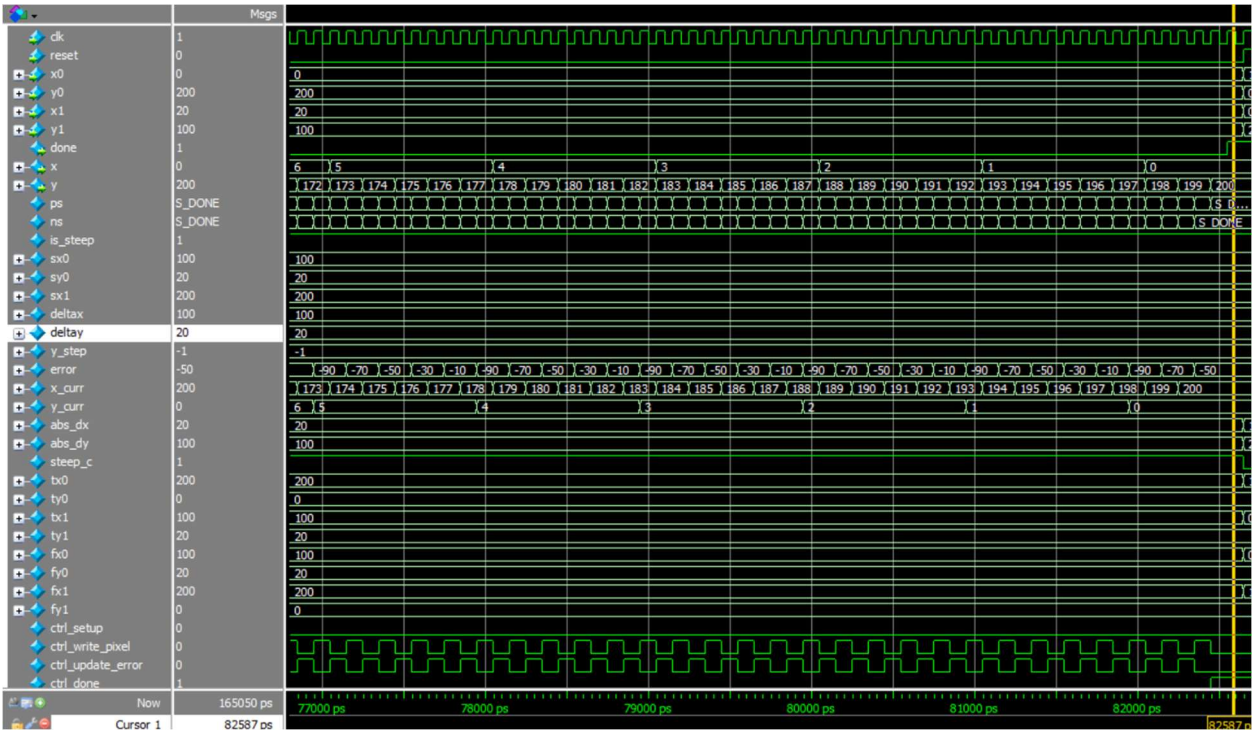


Figure 16: Case 4: Up Right Steep

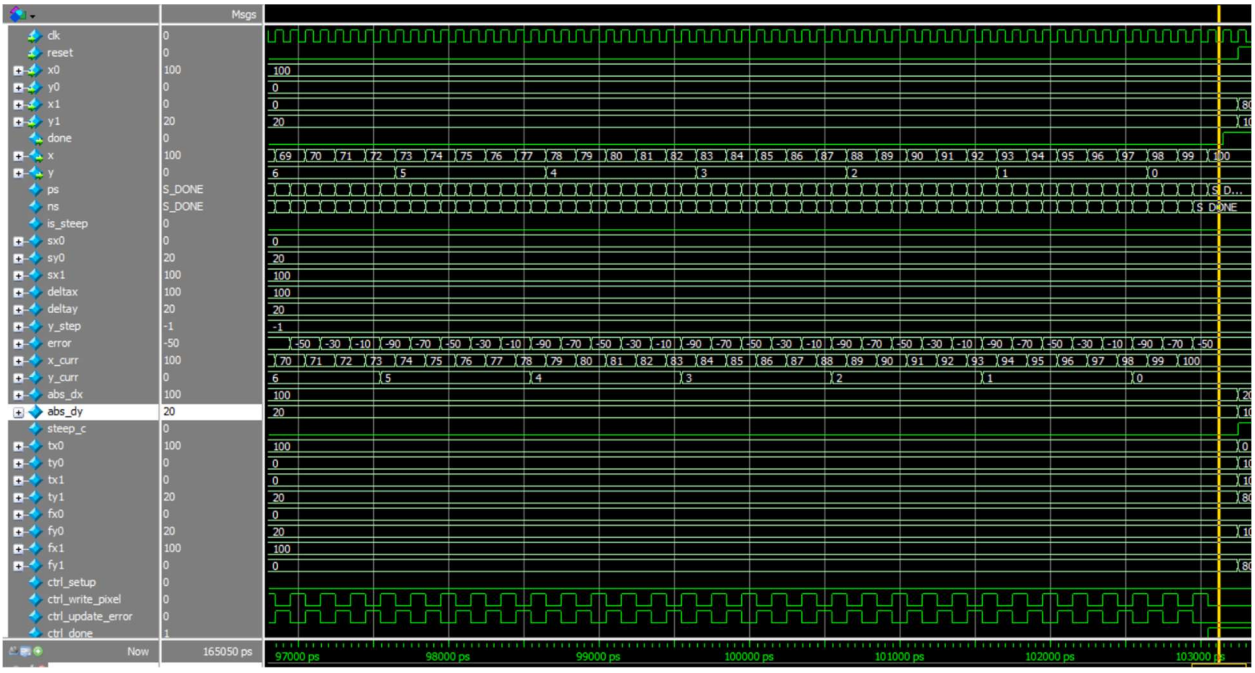


Figure 17: Case 5: Down Left Gradual

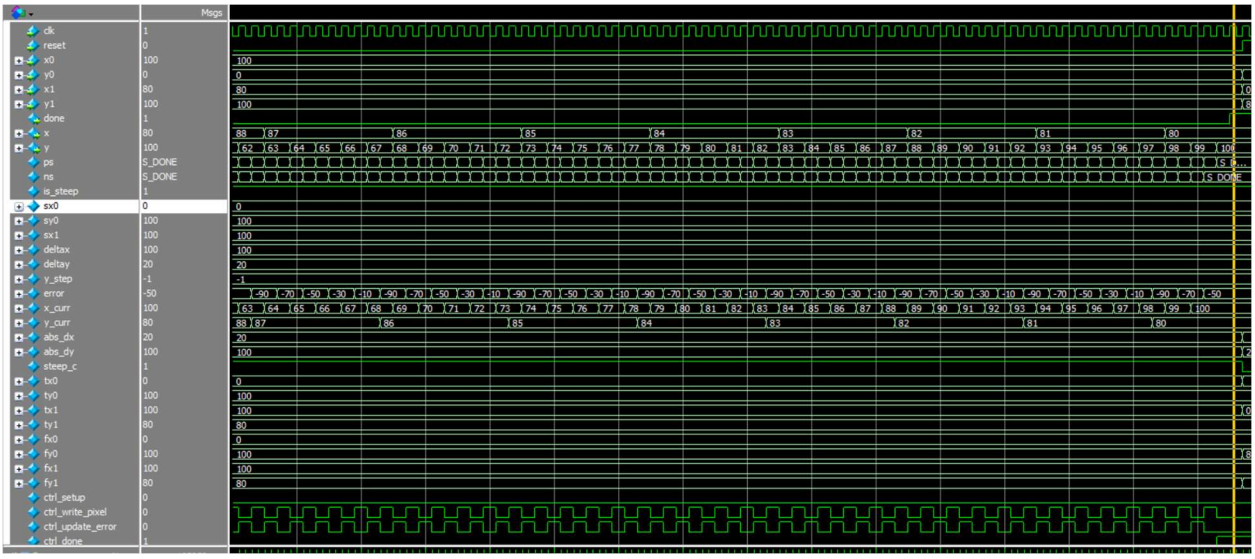


Figure 18: Case 6: Down Left Steep

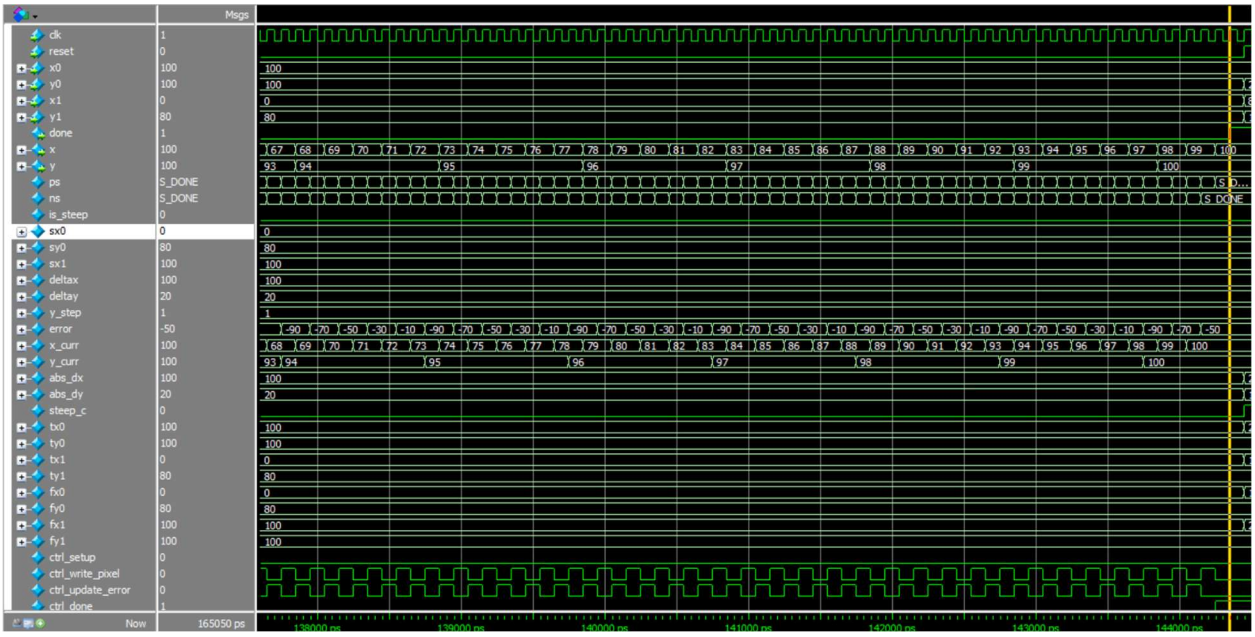


Figure 19: Case 7: Up Left Gradual

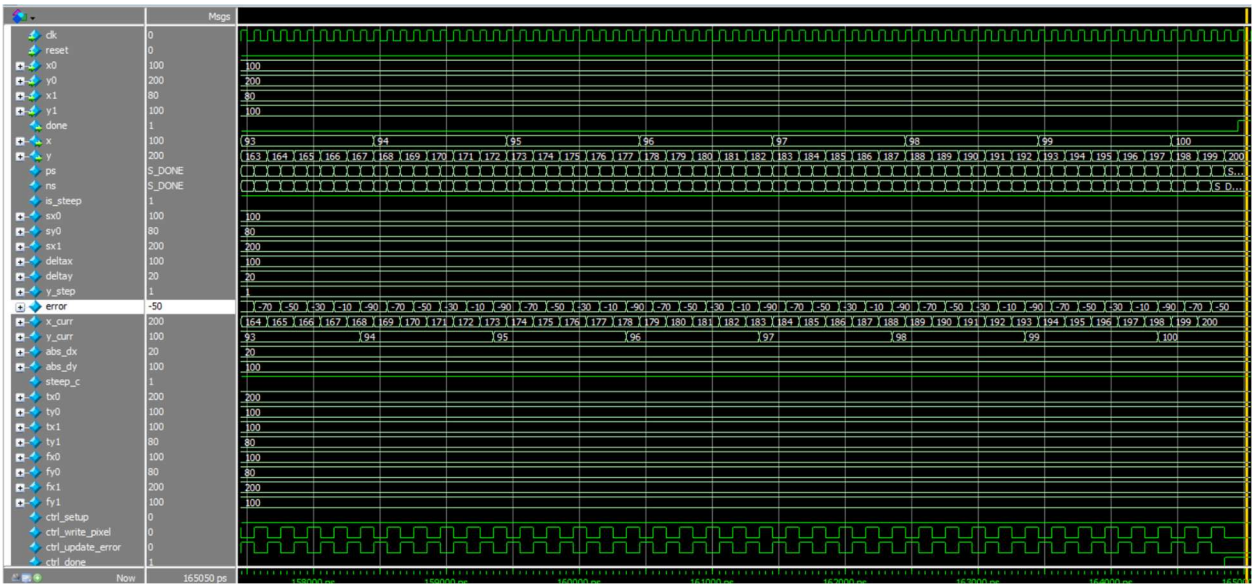


Figure 20: Case 8: Up Right Steep

DE1_SoC

The ASMD for the top level module's implementation is shown in figure 8 below. There were many floating components such as the VGA framebuffer connections, each drawing module, and even the sound selector module. The S_CLEAR, S_WAIT, and S_DRAW states all handle the square drawing module, drawing 64 evenly spaced boxes in a 3x16 row-column format. Once that is done, it will transition to the S_PLAYBACK state which will continuously call the Draw_Playhead module and increment the playhead at each bpm. When a button is pressed, the S_SETUP state chooses the correct register and y coordinate corresponding to the KEY number. It will mark the box and then loop back to the playhead incrementing/drawing. The setup and mark both happen on one clock cycle each, so it is not noticeable in the display that it pauses the playhead. Figure 9 shows how each module is connected to the VGA and how it knows what to draw when. Figure 10 shows how the row registers are selected and changed.

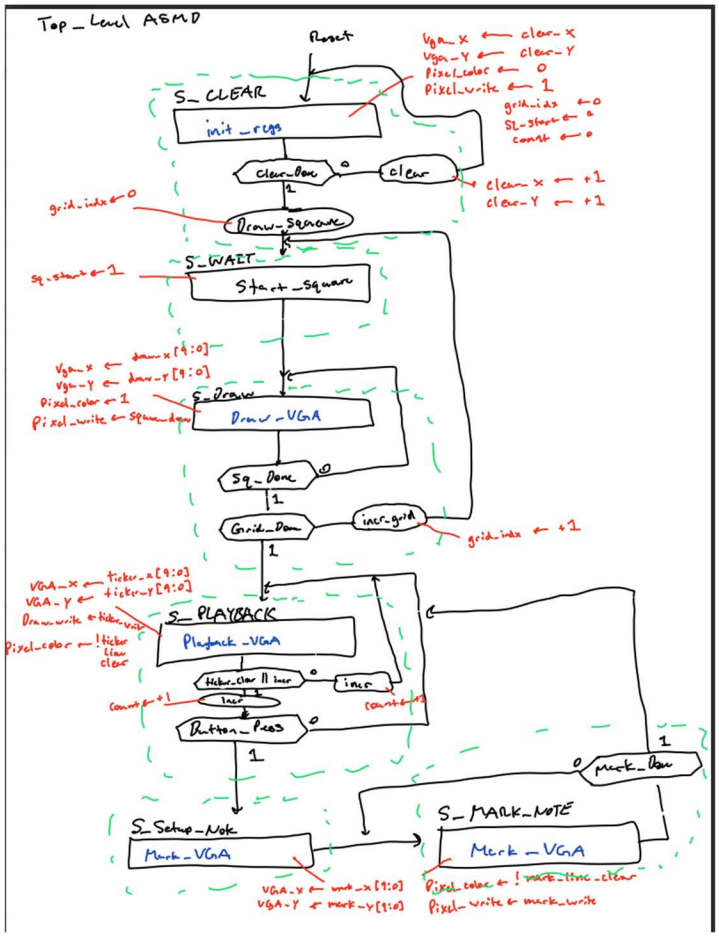


Figure 21: Top Level ASMD

```
// VGA datapath chooses which output to put in the VGA
 always_comb begin
     case (ps)
         S_CLEAR: begin
             vga_x       = clear_x;
             vga_y       = clear_y;
             pixel_color = 1'b0;             // Turn in black
             pixel_write = 1'b1;             // write the black over
         end

         S_DRAW: begin
             vga_x       = draw_x[9:0];          // Draw the square drawer outputs
             vga_y       = draw_y[8:0];
             pixel_color = 1'b1;             // draw it white
             pixel_write = draw_write;       // Write enable from the square module
         end

         S_PLAYBACK: begin
            vga_x = ticker_x[9:0];     //when the playhead wants to be drawn use the playhead outputs for x and y
            vga_y = ticker_y[8:0];
            pixel_write = ticker_write; //whenever the write signal is on from the playhead

            if(ticker_line_clear) begin  //if the line is done and clear is set to high turn the line black
                pixel_color = 1'b0;
            end
            else begin
                pixel_color = 1'b1;      //if the line is being drawn the color is white
            end
         end

         //mark states
         S_SETUP_NOTE, S_MARK_NOTE: begin
            vga_x = mark_x[9:0]; //set the bga drawers to the x and y of the mark module
            vga_y = mark_y[8:0];
            pixel_write = mark_write; //set pixel write to the write wire in mark module
            if(mark_line_clear) begin
                pixel_color = 1'b0; //set color to black
            end
            else pixel_color = 1'b1; //set color to white
         end

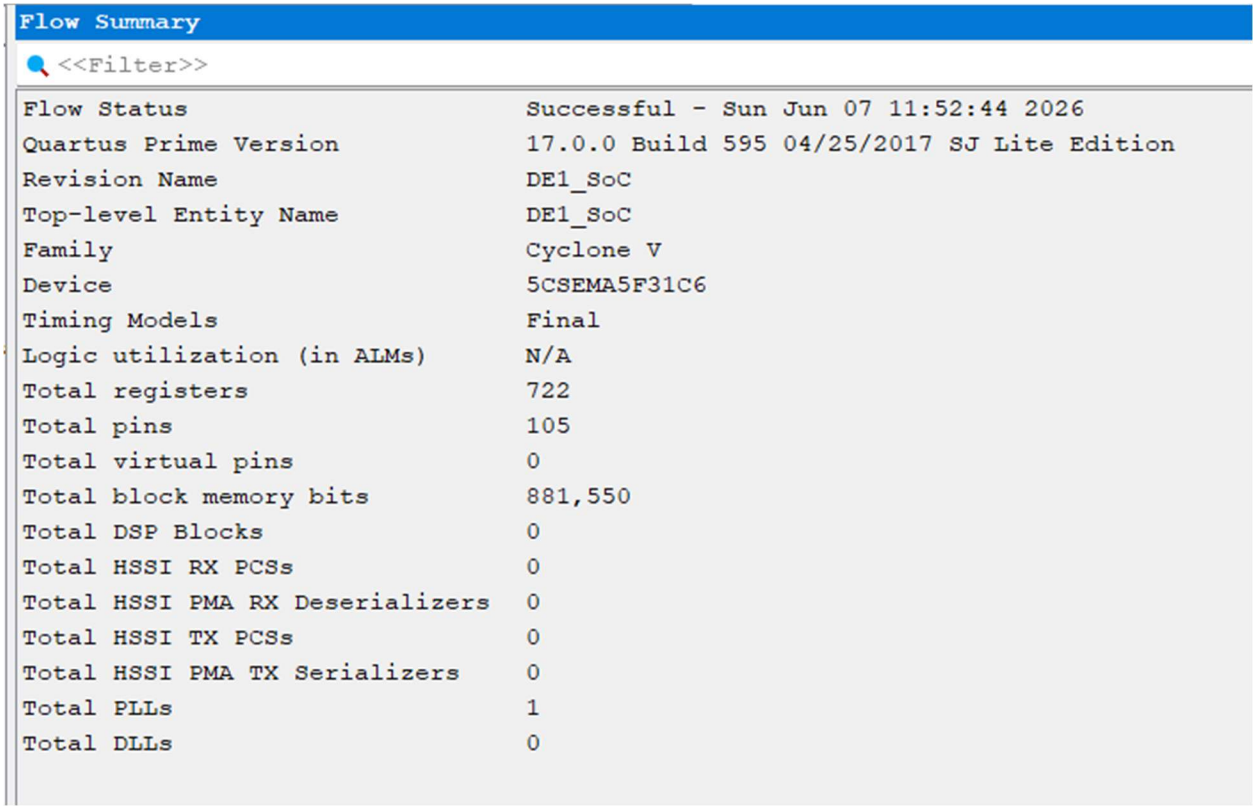
         default: begin //Else everything else turns off
             vga_x       = 10'd0;
             vga_y       = 9'd0;
             pixel_color = 1'b0;
             pixel_write = 1'b0;
         end
     endcase
```

Figure 22: VGA Datapath

```
//flip flop to handle the reset states for the row registers
always_ff @(posedge CLOCK_50) begin
    if (reset) begin
        row0_reg <= 16'd0;
        row1_reg <= 16'd0;
        row2_reg <= 16'd0;
        row3_reg <= 16'd0;
        row_select_reg <= 2'd0;
        note_trigger <= 1'b0;
    end else begin
        if (ps == S_PLAYBACK) begin   //if the playhead is moving in that state
            if(button3Pressed) begin  //given what key is pressed it will choose which row to select
                row_select_reg <= 2'd0;
                note_trigger <= 1'b1;
            end
            else if(button2Pressed) begin
                row_select_reg <= 2'd1;
                note_trigger <= 1'b1;
            end
            else if(button1Pressed) begin
                row_select_reg <= 2'd2;
                note_trigger <= 1'b1;
            end
            else if (button0Pressed) begin
                row_select_reg <= 2'd3;
                note_trigger <= 1'b1;
            end
        end
        if(ps == S_MARK_NOTE && mark_done) begin //if we are in the mark note state and the mark is done update the registers
            note_trigger <= 1'b0;
            if(row_select_reg == 2'd0) row0_reg[playhead_count] <= ~row0_reg[playhead_count];
            if(row_select_reg == 2'd1) row1_reg[playhead_count] <= ~row1_reg[playhead_count];
            if(row_select_reg == 2'd2) row2_reg[playhead_count] <= ~row2_reg[playhead_count];
            if(row_select_reg == 2'd3) row3_reg[playhead_count] <= ~row3_reg[playhead_count];
        end
    end
end
```

Figure 23: Row registers for each note and how they change based on KEY inputs

Flow Summary



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Jun 07 11:52:44 2026
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	DE1_SoC
Top-level Entity Name	DE1_SoC
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	722
Total pins	105
Total virtual pins	0
Total block memory bits	881,550
Total DSP Blocks	0
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1
Total DLLs	0

Figure 24: Flow Summary

Experience Report

This project was the most entertaining and fulfilling. Although it took by far the longest out of any labs we have completed this quarter, it did not feel like it. Designing, planning, and implementing our ideas from the start all the way to the end made it more engaging and fulfilling. With that said the process was anything but smooth. A lot of our time went into designing and understanding how we wanted to tackle the sound

Our hardest challenge was definitely combining everything into the top-level De1_SoC module and creating an ASMD for it. There were a lot of timing errors especially with regarding the Mark_Play and Playhead modules. In order to fix them, we included more states and internal start/reset signals so that the functionality worked as intended.

- Reading – 1hr
- Planning – 5hrs
- Design – 7hrs
- Coding – 15hrs
- Testing – 5hrs

- Debugging – 7hrs